ENGINEERING ARCHITECTURE SPECIFICATION • CA3 DELIVERABLE

RegRadar: System Design Document

Autonomous Multi-Agent Regulatory Compliance Radar with Adaptive Adversarial Validation

System Name: RegRadar v1.0.0	**Architecture Pattern:** LangGraph Multi-Agent StateGraph	**Target Environment:** Local & Vercel Serverless
Backend: Python 3.12 / FastAPI	**Frontend:** React 19 / TypeScript / Vite	**Persistence:** SQLite (Local) / Neon PostgreSQL

1. SYSTEM OVERVIEW & ARCHITECTURAL OBJECTIVES

RegRadar is designed to autonomously translate unstructured regulatory circulars into calibrated, verifiable enterprise compliance gaps. The system replaces error-prone, single-pass LLM prompts with a deterministic-governed multi-agent pipeline featuring an adversarial feedback loop.

Primary Design Objectives:

- **Decoupled Agent Specialisation:** Six agents with strict single-responsibility boundaries communicate exclusively via a validated shared state.
- **Deterministic Authority over Non-Deterministic Models:** LLMs are strictly leveraged for linguistic extraction and synthesis. In contrast, control mapping, parameter verification, and validation rules are deterministic, reproducible, and explainable.
- **Adversarial Feedback Loop:** An independent Novelty Validator challenges every gap proposal. The synthesiser must programmatically resolve, withdraw, or defend rejections.
- **Finite Bounded Convergence:** Guaranteed loop termination via state-tracked revision limits and automated human escalation.

2. STATE ARCHITECTURE & COMMUNICATION PROTOCOL

Agents never invoke each other directly. The graph executes over a strongly typed LangGraph `RegRadarState` (defined in `backend/orchestration/state.py`).

2.1 State Definition & Reducer Mechanics

```
class RegRadarState(TypedDict):
    run_id: str
    mode: Literal["demo", "live"]
    request: dict[str, Any]
    analysis_date: str
    documents: list[dict[str, Any]]
    obligations: list[dict[str, Any]]
    coverage: list[dict[str, Any]]
    gaps: list[dict[str, Any]]
    pending_feedback: list[dict[str, Any]]
    validation_decisions: list[dict[str, Any]]
    validation_round: int
    revision_count: int
    max_revisions: int
    remediation_plan: list[dict[str, Any]]
    report: dict[str, Any]
    status: Literal["running", "completed", "failed"]
    trace: Annotated[list[dict[str, Any]], operator.add]         # Additive append
    errors: Annotated[list[str], operator.add]                   # Additive append
    route_history: Annotated[list[str], operator.add]            # Additive append
```

All operational keys are replaced by the writing agent. Keys tracking auditability (`trace`, `errors`, `route_history`) utilize the `operator.add` reducer, ensuring an immutable chronological log of every inter-agent message. Non-

serializable dependencies (LLM clients, application settings) are decoupled into a thread-safe `RunContext`.

3. AGENT DETAILED SPECIFICATIONS

Agent	Inputs	Outputs	Key Algorithms & Decision Logic
Regulatory Scout	`request`	`documents`	Retrieves circulars from synthetic corpus, uploaded texts, or live RBI/SEBI RSS feeds. Computes domain relevance against organization themes: $$\text{Relevance} = \frac{\sum_{t \in \text{Tokens}} \mathbb{I}(t \in \text{Themes})}{\|\text{Tokens}\|}$$
Obligation Extractor	`documents`	`obligations`	Splits clauses into sentences; applies deontic modal analysis: • *shall / must / required* → `mandatory` • *should / recommended* → `recommended` • *may / optional* → `permissive` Regex parameter extractor parses `deadline_hours`, `retention_days`, and `frequency`. *Anti-Hallucination Guard:* Every proposed sentence is validated to appear verbatim in the source clause.
Control Mapper	`obligations`, Control Library	`coverage`	Constructs TF-IDF index across control library and obligations. Similarity score: $S = 0.65 \times \text{Cosine}(v_o, v_c) + 0.35 \times \text{KeywordOverlap}(T_o, K_c)$. Categorization: ≥ 0.42 = Covered (if no shortfalls); ≥ 0.20 = Partial; < 0.20 = Gap. Automated quantitative shortfall evaluation (e.g. 24h internal SLA vs 6h regulatory requirement).
Gap Synthesiser	`coverage`, `pending_feedback`	`gaps`	**Draft Mode (Round 0):** Synthesizes actionable gaps for partial/uncovered obligations. **Revise Mode (Round 1+):** Evaluates validator feedback. For each rejected gap, executes one of: `revise` (updates fields), `withdraw` (accepts invalidity/prior tracking), or `maintain` (defends position).
Novelty Validator	`gaps`, `coverage`, Known Issues Register	`validation_decisions`, `pending_feedback`	Adversarial auditor. Independently evaluates 9 deterministic rules. If any check fails, emits a blocking `ValidationIssue` contract containing `code`, `field`, `expected`, and `message`.
Output Agent	State aggregate	`remediation_plan`, `report`	Computes multi-factor risk scores, assigns priority tiers (P1/P2/P3) and SLAs, designates departmental owners (CISO, DPO, CRO), and compiles executive Markdown report.

4. THE NINE AUTHORITATIVE VALIDATION CHECKS

The Novelty Validator enforces 9 independent rules. The validator re-derives all facts from original inputs and the Known Issues Register, preventing sycophantic acceptance:

#	Check Name	Rejection Code	Deterministic Failure Condition
1	`obligation_exists`	`unknown_obligation`	The gap references an obligation ID not present in the current run's extracted obligations.
2	`required_detail`	`insufficient_detail`	Title length < 8 chars, description length < 40 chars, or recommended action length < 15 chars.
3	`citation_consistency`	`citation_mismatch`	Gap citation does not match the true obligation source citation.
4	`evidence_grounding`	`evidence_not_grounded`	Evidence quote is < 20 characters or is not a verbatim substring of the cited obligation text.
5	`not_already_covered`	`already_covered`	The coverage matrix indicates that an existing control already covers the obligation without shortfalls.
6	`novel_vs_known_issues`	`already_tracked`	The obligation citation matches an entry in the organization's Known Issues Register.
7	`not_duplicate`	`duplicate_proposal`	Another accepted or earlier pending proposal in the current run addresses the same obligation.
8	`mandatory_classification`	`non_mandatory_obligation`	A gap is classified as `mandatory` when the underlying clause modal is non-mandatory (*should*/*may*).
9	`severity_calibration`	`severity_understated`	Proposed severity is lower than policy minimums (e.g., regulatory reporting ≤ 6h must be `critical`).

5. GRAPH ROUTING & ADAPTIVE FEEDBACK LOOP DYNAMICS

The state machine dynamically routes execution after the Novelty Validator finishes evaluating proposals:

```
function route_after_validation(state):
  rejected_gaps = [g for g in state.gaps if g.status == "rejected"]
  if len(rejected_gaps) == 0: return "finalise"                    # → Output Agent
  if state.revision_count < state.max_revisions: return "revise"     # → Gap Synthesiser
  return "escalate"                                                   # → Escalation Node
```

5.1 Escalation Node Mechanics

When an impasse occurs (rejections remain when `revision_count == max_revisions`), the **Escalation Node** marks unresolved gaps as `escalated` and emits a high-priority message to the recipient `human_reviewer`. The message includes the gap definition, the validator's rejection issues, and the synthesiser's defensive counter-argument, ensuring human oversight without stalling the pipeline.

6. REMEDIATION PRIORITISATION & MATHEMATICAL RISK FORMULA

The Output Agent calculates risk scores and priority tiers for all accepted gaps using the formula:

```
Risk Score = Severity Weight × Urgency Factor × Exposure Factor × Classification Factor
```

- **Severity Weight (S_w):** `critical` = 10.0, `high` = 7.0, `medium` = 4.0, `low` = 2.0.
- **Urgency Factor (U_f):** Effective date past/today = 1.6; ≤ 30 days = 1.4; ≤ 90 days = 1.2; > 90 days = 1.0; Unknown = 1.1.

- **Exposure Factor (E_f):** 1.25 for high-impact domains (`incident reporting` , `data protection`); 1.0 otherwise.
- **Classification Factor (C_f):** `mandatory` = 1.0; `advisory` = 0.5.
- **Priority Tier & SLA:**
  - **P1 (Critical):** Risk Score ≥ 10.0 → 30-day resolution SLA.
  - **P2 (Medium):** $5.0 \le \text{Risk Score} < 10.0$ → 60-day resolution SLA.
  - **P3 (Low):** Risk Score < 5.0 → 90-day resolution SLA.

7. DATA PERSISTENCE & STORAGE ARCHITECTURE

RegRadar maintains complete run histories across two primary relational tables (SQLite in local development; PostgreSQL/Neon in cloud serverless deployments):

- `runs` **table:** Stores `id` , `created_at` , `completed_at` , `status` , `mode` , `llm_provider` , `llm_model` , `request_json` , `stats_json` , `result_json` , and `error` .
- `messages` **table:** Stores the granular execution trace. Each row represents an `AgentMessage` with `id` , `run_id` , `seq` , `timestamp` , `sender` , `recipient` , `message_type` , `summary` , `payload_json` , `revision_round` , `duration_ms` , and `llm_usage_json` .

8. QUALITY VERIFICATION & TEST PROOF

The repository contains 64 automated pytest tests verifying every subsystem:

- `tests/test_text_and_parsing.py` : Verifies sentence splitting, modal verb extraction, parameter regex, and TF-IDF cosine calculations.
- `tests/test_agents.py` : Unit tests all six agents and individual validation checks.
- `tests/test_feedback_loop.py` : Formally tests the rejection → structured feedback → revision → re-validation sequence.
- `tests/test_api.py` : Tests all 18 REST endpoints, CORS headers, error status codes, and JSON schemas.